

64) WAP to implement single link list with to simulate stack and Queue operations.

```
createNode()  
create new node  
set node.data = value  
set node.next = null  
return node
```

Stack operations:

push()

```
newnode = createNode(value)  
newnode → next = top  
top = newnode
```

printf("pushed element").

pop()

if top = null

printf("empty")

return

temp = top

printf("popped element")

top = top → next

free(temp).

display()

if top == null

printf("empty")

return

temp = top

printf("stack elements: ")

while (temp != null)

printf(temp → data)

temp = temp → next.

Queue operations:

enqueue()

newnode = createNode(value)

if rear == null

front = rear = newnode

rear → next = newnode

rear = newnode

printf("enqueued element")

dequeue()

if front == null

printf("empty")

return

temp = front

printf("dequeued element")

front = front → next

if front == null

rear = null

free(temp)

display()

if front = null

printf("empty")

return

temp = front

printf("queue elements")

while (temp != null)

printf(temp → data)

temp = temp → next.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct Node *top = NULL;
```

```
struct Node *front = NULL;
```

```
struct Node *rear = NULL;
```

```
struct Node* createNode(int value){
```

```
    struct Node *newNode = (struct Node*) malloc (sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    return newNode;
```

```
}
```

```
void push(int value){
```

```
    struct Node *newNode = createNode(value);
```

```
    newNode->next = top;
```

```
    top = newNode;
```

```
    printf("%d pushed to stack.\n", value);
```

```
}
```

```
void pop(){
```

```
    if(top == NULL){
```

```
        printf("Stack is empty. Nothing to pop.\n");
```

```
        return;
```

```
}
```

```
    struct Node *temp = top;
```

```
    printf("%d popped from stack.\n", top->data);
```

```
    top = top->next;
```

```
    free(temp);
```

```
}
```

```
void displayStack(){
```

```
    struct Node *temp = top;
```

```
    if(temp == NULL){
```

```
        printf("Stack is empty.\n");
```

```
        return;
```

```
}
```

```
printf("stack (Top to bottom):");
```

```
while (temp != NULL)
```

```
printf("%d", temp->data);
```

```
temp = temp->next;
```

```
}  
printf("\n");
```

```
}  
void enqueue(int value) {
```

```
struct Node *newNode = createNode(value);
```

```
if (rear == NULL) {
```

```
front = rear = newNode;
```

```
} else {
```

```
rear->next = newNode;
```

```
rear = newNode;
```

```
{  
printf("%d enqueued to the queue\n", value);
```

```
}
```

```
void dequeue() {
```

```
if (front == NULL) {
```

```
printf("Queue is empty. Nothing to dequeue.\n");
```

```
return;
```

```
{  
struct Node *temp = front;
```

```
printf("%d dequeued from the queue\n", temp->data);
```

```
front = front->next;
```

```
if (front == NULL)
```

```
rear = NULL;
```

```
free(temp);
```

```
}
```

```
void displayQueue() {
```

```
struct Node *temp = front;
```

```
if (temp == NULL) {
```

```
printf("Queue is empty.\n");
```

```
return;
```

```
{
```

```
printf("Queue (Front to Rear):");
```

```
while (temp != NULL) {
```

```
printf("%d", temp->data);
```

```

temp = temp -> next;
}
printf("\n");
}

int main() {
    int choice, value, ch;
    while(1) {
        printf("\n-- Singly linked list Simulation --\n");
        printf("1. Stack Operations\n");
        printf("2. Queue Operations\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                while(1) {
                    printf("\n-- Stack Menu --\n");
                    printf("1. push\n");
                    printf("2. pop\n");
                    printf("3. Display stack\n");
                    printf("4. Back to Main menu\n");
                    printf("Enter your choice: ");
                    scanf("%d", &ch);
                    switch(ch) {
                        case 1:
                            printf("Enter value to push: ");
                            scanf("%d", &value);
                            push(value);
                            break;
                        case 2:
                            pop();
                            break;
                        case 3:
                            displaystack();
                            break;
                        case 4:
                            goto main_menu;
                    }
                }
            case 2:
                // Queue operations
            case 3:
                // Exit
            case 4:
                // Back to Main menu
        }
    }
}

```

default:

printf("Invalid choice.\n");

}

}

break;

case 2:

while(1){

printf("In--Queue Menu--\n");

printf("1. Enqueue.\n");

printf("2. Dequeue.\n");

printf("3. Display Queue");

printf("4. Back to main menu.\n");

printf("Enter your choice: ");

scanf("%d", &ch);

switch(ch){

case 1:

printf("Enter value to enqueue: ");

scanf("%d", &value);

enqueue(value);

break;

case 2:

dequeue();

break;

case 3:

displayQueue();

break;

case 4:

goto main-menu;

default:

printf("Invalid choice.\n");

}

break;

case 3:

printf("Exiting Program.\n");

exit(0);

default:

printf("Invalid choice. Try again.\n");

}

main-menu :

return 0;

4.

olp

--- Singly linked list simulation ---

1. Stack operations

2. Queue operations

3. Exit

Enter your choice: 1

--- Stack menu ---

1. push

2. pop

3. Display stack

4. Back to main menu

Enter your choice: 1

Enter value to push: 10

10 pushed to stack.

--- Stack menu ---

1. push

2. pop

3. Display stack

4. Back to main menu

Enter your choice: 1

Enter value to push: 20

20 pushed to stack

--- Stack menu ---

1. push

2. pop

3. Display stack

4. Back to main menu

Enter your choice: 2

20 popped from stack

--- Stack menu ---

1. push

2. pop

3. Display stack

4. Back to main menu

Enter your choice: 3

Stack (Top to Bottom): 10

--- Stack menu ---

1. push

2. pop

3. Display stack

4. Back to main menu

Enter your choice: 4.

-- Singly linked list simulation --

1. Stack Operations
2. Queue Operations
3. Exit

Enter your choice: 2

-- Queue Menu --

1. Enqueue
2. Dequeue
3. Display Queue
4. Back to main menu

Enter your choice: 1

Enter value to enqueue: 10

10 enqueued to the queue

-- Queue Menu --

1. Enqueue
2. Dequeue
3. Display Queue
4. Back to main menu

Enter your choice: 1

Enter value to enqueue: 20

20 enqueued to the queue

-- Queue Menu --

1. Enqueue
2. Dequeue
3. Display Queue
4. Back to main menu

Enter your choice: 1

Enter value to enqueue: 30

30 enqueued to the queue.

-- Queue Menu --

1. Enqueue
2. Dequeue
3. Display Queue
4. Back to main menu

Enter your choice: 2

10 dequeued from the queue.

-- Queue Menu --

1. Enqueue
2. Dequeue
3. Display Queue
4. Back to main menu

-- Singly linked list simulation --

1. Stack Operations

2. Queue Operations

3. Exit

Enter your choice: 3

Exiting program.